

PRD

DOODLEBOT

Product Requirements Document (PRD)

IEEE Ahmedabad University Student Branch — Club Carnival

1. OVERVIEW

DoodleBot is a browser-based, AI-powered Pictionary game built for Club Carnival. Participants scan an IEEE-branded QR code, draw a randomly assigned word on their own phone, and watch a computer vision model try to guess it in real time. Scores feed a live leaderboard. The product's job is to pull footfall to the IEEE stall and hold attention there, repeatedly, while visibly demonstrating a working AI system.

1.1 Problem Statement

Static stalls and posters don't hold attention at a crowded carnival. IEEE needs an attraction that (a) draws a crowd from a distance, (b) gives people a reason to stay and replay, and (c) actually demonstrates technical competence rather than just claiming it.

1.2 Objective

Build a zero-install, phone-based game where an AI guesses user sketches in real time, wrapped entirely in IEEE branding, with a competitive leaderboard driving repeat play.

1.3 Target Users

- Primary: Carnival attendees (students, mostly non-technical), walking past the IEEE stall
- Secondary: IEEE members/volunteers running the stall, monitoring the leaderboard screen
- Tertiary: IEEE committee reviewing this as a technical showcase

2. GOALS AND SUCCESS METRICS

Goals

- Drive footfall and dwell time at the IEEE stall
- Demonstrate a real, working AI/computer vision use case
- Reinforce IEEE brand identity through repeated visual exposure
- Be trivially easy to join (scan → play, under 10 seconds)

Success Metrics

- Total unique participants (target: 500–600 footfall exposure)
- Total games played (repeat play rate is the key signal — target 1.5–2x plays per participant)
- Concurrent session count during peak (target: handle 20–30 simultaneous players without degradation)
- Leaderboard engagement (screen viewed/checked by non-players, informal observation)

Out of Scope (v1)

- User accounts / login / persistent profiles across events
- Native mobile app
- Multiplayer head-to-head drawing (same round, competing live)
- Prize/reward fulfillment logic (handled manually by volunteers, not the product)
- Any drawing data retention beyond leaderboard stats (no need to store sketch images)

3. USER STORIES

- As a participant, I want to scan a QR code and start playing within seconds, without installing anything.
- As a participant, I want to pick from a few word options so I have some control over what I draw.
- As a participant, I want to see the AI's guesses update live while I draw, so the game feels responsive and "alive."
- As a participant, I want to know immediately whether I won, how fast, and where I rank.
- As a participant, I want to replay instantly without re-scanning or re-entering my name from scratch (or a fast re-entry).
- As a volunteer, I want a leaderboard visible on a stall screen that updates live, to draw a crowd.
- As a committee reviewer, I want to see that the system is technically sound, brand-compliant, and low-risk to run live.

4. FUNCTIONAL REQUIREMENTS

4.1 Entry & Onboarding

- QR code deep-links to the game's landing page (mobile web, responsive).
- Landing page carries IEEE branding and a one-line explanation of the game.
- Name entry: single text field, client-side validated (non-empty, reasonable length, basic profanity filter recommended).

4.2 Word Selection

- On starting, the system presents 3 randomly selected words from a curated word bank.
- Word bank should be scoped to sketch-recognizable, carnival-appropriate objects (see Architecture doc for word bank design).
- Participant selects one; selection locks in the round.

4.3 Countdown & Drawing

- 3-second countdown before the canvas becomes active.
- Drawing canvas: single-color or minimal palette, touch/mouse-based freehand drawing, undo/clear optional but not required for v1.

- Round timer: configurable 90–120 seconds, visibly counting down.

4.4 Real-Time AI Guessing

- As the participant draws, the canvas is periodically sampled and sent to the recognition model.
- The model returns ranked candidate classes with confidence scores.
- The UI displays the current top guess, updating as new predictions arrive.
- Round ends immediately on a correct match, or when the timer expires.

4.5 Results

- On round end, show: correct/incorrect, time taken (if correct), and current leaderboard position.
- Provide a clear "Play Again" action that returns to word selection (name can persist for the session).

4.6 Leaderboard

- Publicly viewable page (intended for a stall-facing screen/monitor).
- Metrics: fastest overall guess, fastest guess per word, most successful AI guesses (per participant), total games played, overall ranking.
- Leaderboard updates live or near-live (a few seconds' delay is acceptable).

4.7 Branding

- IEEE logo, color palette, and identity applied consistently across landing page, game screen, results screen, and leaderboard (see UX/UI Design Bible).

5. NON-FUNCTIONAL REQUIREMENTS

- Performance: must comfortably support 20–30 concurrent active sessions without perceptible lag in AI prediction updates.
- Compatibility: must work on typical Android/iOS mobile browsers (Chrome, Safari) without installation.
- Reliability: game must degrade gracefully on poor stall Wi-Fi (e.g., retry logic, offline-tolerant canvas capture).
- Latency: AI prediction round-trip should feel "live" — target under ~1 second per prediction cycle.
- Data handling: no sketch images retained after prediction; only scores/metadata persisted.
- Accessibility: legible type and sufficient contrast per IEEE palette even in bright outdoor/indoor carnival lighting.

6. RISKS AND MITIGATIONS

- Poor/unstable stall internet → mitigate with lightweight payloads, retry/backoff, and a wired fallback connection if available.

- AI model fails to recognize valid sketches → curate a word bank matched to the model's known-strong classes; test extensively beforehand.
- High concurrency causing slow predictions → load-test beforehand; consider client-side model inference to avoid server bottlenecks (see Architecture doc).
- Inappropriate names/content entered by participants → basic client-side filtering; volunteer can moderate leaderboard if needed.
- Device/browser fragmentation → test on common device set (iOS Safari, Android Chrome) before the event.

7. TIMELINE (SUGGESTED)

- Week 1: Finalize word bank, model selection, branding assets
- Week 2: Build core game flow (landing → draw → result)
- Week 3: Build leaderboard + backend, integrate AI prediction loop
- Week 4: Branding polish, load testing, dry run with volunteers
- Event day: Deploy, monitor, iterate live if needed

(Adjust against actual Club Carnival date.)

8. OPEN QUESTIONS

- Final word bank list and count (recommend 40–60 words for variety without diluting recognition accuracy)
- Exact round timer (90s vs 120s) — recommend deciding after playtesting
- Whether leaderboard resets per day or runs cumulatively across the full carnival
- Physical setup: is there a dedicated screen for the leaderboard, or is it phone-only?

System Architecture & Technical Design

DOODLEBOT

System Architecture & Technical Design

IEEE Ahmedabad University Student Branch — Club Carnival

1. HIGH-LEVEL ARCHITECTURE

DoodleBot is a client-heavy web application: most of the interactive experience (drawing, live AI guessing) should run in the browser to minimize server load and latency, with a lightweight backend used only for persistence (leaderboard data) and serving static assets.

Components:

1. Client (Mobile Web App) — landing page, name entry, word selection, drawing canvas, live AI inference, results screen
2. AI Recognition Model — sketch classification model, ideally run client-side (in-browser) via a lightweight ML runtime; server-side inference as fallback
3. Backend API — thin service for leaderboard reads/writes and session/game metadata
4. Database — stores participants (name, session), game results, aggregated leaderboard stats
5. Leaderboard Display — same web app, a dedicated route optimized for a stall-facing screen, auto-refreshing

Text Diagram — System Flow:

[Participant Phone]

- scans QR code
- loads Landing Page (Client App)
- enters name → sent to Backend API → stored in DB (Participants)
- selects word (client-generated random 3-word set from local word bank)
- Drawing Canvas active
 - canvas snapshot sampled every ~300-500ms
 - snapshot passed to AI Recognition Model (runs in-browser)
 - model returns ranked guesses + confidence scores
 - top guess displayed live in UI
- on correct guess OR timer expiry:
 - result computed client-side (correct?, time taken)
 - result posted to Backend API → stored in DB (Games/Results)
 - Backend recalculates/reads aggregated leaderboard stats
- Results Screen shown → "Play Again" loops back to Word Selection

[Stall Display Screen]

- loads Leaderboard Route (Client App)
- polls/subscribes to Backend API for leaderboard data
- auto-refreshes on interval (e.g., every 5-10s) or via lightweight real-time channel

2. WHY CLIENT-SIDE AI INFERENCE

Running the sketch recognition model in-browser (rather than sending sketch data to a server for every prediction) is strongly recommended:

- Avoids server bottlenecks at 20-30 concurrent users each sending predictions multiple times per second
- Removes dependency on stall internet bandwidth for the core gameplay loop (only final results need to reach the server)
- Reduces latency — predictions feel instant since there's no network round-trip
- Sketch data never needs to leave the device, simplifying privacy/data-handling

A pre-trained sketch classification model (trained on a large open sketch dataset covering common object categories) can be exported to a browser-friendly format and loaded once at page load. The prediction loop then runs entirely client-side against the curated word bank's corresponding classes.

Fallback: if client-side inference proves infeasible on lower-end devices, a lightweight server-side inference endpoint can be used instead, with snapshot sampling throttled to reduce load.

3. WORD BANK DESIGN

- The word bank should be restricted to object classes the recognition model handles reliably (avoid obscure/ambiguous categories).
- Recommend 40-60 words, grouped loosely by difficulty (easy/medium/hard) so word selection can optionally be balanced.
- Each word maps to one or more valid model class labels (some words may have multiple acceptable matches, e.g., "couch" / "sofa").
- Word bank stored as a static config (JSON) on the client — no need for a database round-trip to fetch words.

4. DATA MODEL (LEADERBOARD BACKEND)

Entities:

Participant

- id
- name
- created_at

GameResult

- id
- participant_id (FK)
- word
- correct (boolean)
- time_taken_seconds (nullable if incorrect)
- created_at

Derived / Aggregated (computed, not necessarily stored directly):

- Fastest overall guess → $\text{MIN}(\text{time_taken_seconds}) \text{ WHERE correct} = \text{true}$
- Fastest guess per word → $\text{MIN}(\text{time_taken_seconds}) \text{ GROUP BY word WHERE correct} = \text{true}$
- Most successful AI guesses per participant → $\text{COUNT}(\ast) \text{ GROUP BY participant_id WHERE correct} = \text{true}$
- Total games played → $\text{COUNT}(\ast)$ on GameResult (overall or per participant)
- Overall ranking → ORDER BY successful guesses DESC, average time ASC (exact ranking formula to be finalized based on desired incentive — see Section 6)

A simple relational database (e.g., Postgres, or even a lightweight hosted option like SQLite/Firebase/Supabase) is sufficient at this scale — no need for complex infrastructure.

5. API SURFACE (INDICATIVE)

POST /participants — create participant { name } → { participant_id }

POST /games — submit a game result { participant_id, word, correct, time_taken_seconds } → { game_id }

GET /leaderboard — returns aggregated leaderboard data (all metrics needed for the leaderboard page)

GET /leaderboard/word/{word} — fastest guess for a specific word (optional, if per-word view is shown)

Keep the API surface minimal — this is a two-write, one-read system at its core.

6. RANKING LOGIC (RECOMMENDATION)

To keep the leaderboard meaningful and to encourage repeat play (not just one lucky fast guess), recommend a composite score rather than a single raw metric, e.g.:

$$\text{score} = (\text{successful_guesses} \times \text{weight_a}) + (\text{speed_bonus} \times \text{weight_b})$$

Where speed_bonus rewards faster average correct-guess times. Exact weights should be

tuned during playtesting so that both "played a lot and did well" and "got a few very fast guesses" feel fairly rewarded. This avoids a leaderboard dominated purely by whoever played the most times.

7. TECH STACK (SUGGESTED — NOT PRESCRIPTIVE)

- Frontend: Any modern web framework capable of canvas drawing + client-side ML inference (e.g., a React-based app with an in-browser ML runtime for sketch recognition)
- AI Model: A pre-trained sketch-recognition model exported to a browser-compatible inference format
- Backend: Lightweight API service (Node/Express, or a managed backend-as-a-service to reduce build time given the short timeline)
- Database: Managed hosted database (to avoid self-hosting/ops burden during a live event)
- Hosting: Any static-friendly hosting provider with a custom subdomain under IEEE branding (e.g., doodlebot.ieeeau.org or similar), HTTPS required (QR/camera-adjacent flows expect secure origins)

The exact stack is flexible; the two hard constraints are (1) client-side inference for the drawing loop, and (2) a minimal, low-maintenance backend given this is a short-lived event tool.

8. INFRASTRUCTURE & RELIABILITY FOR EVENT DAY

- Deploy well before the event; freeze changes 24-48 hours prior except critical fixes.
- Load-test with a simulated 20-30 concurrent sessions before the event.
- Have a fallback plan for internet loss at the stall: since drawing/inference is client-side, only leaderboard sync is affected — results can queue locally and sync when connectivity resumes.
- Assign one volunteer/technical owner to monitor the backend/leaderboard during the event and restart services if needed.

9. FLOWCHART — PARTICIPANT SESSION (STATE MACHINE VIEW)

[Landing] → [Name Entry] → [Word Selection] → [Countdown] → [Drawing + Live AI Guessing]
→ (correct guess) → [Result: Win] → [Play Again?] → back to [Word Selection]
→ (timer expires) → [Result: No Match] → [Play Again?] → back to [Word Selection]

Each transition is a simple, linear state — there is no branching complexity beyond win/lose, which keeps both the implementation and the on-device performance profile lightweight.

UX/UI Design Bible

DOODLEBOT

UX/UI Design Bible

IEEE Ahmedabad University Student Branch — Club Carnival

1. DESIGN PRINCIPLES

- Zero-friction entry: from QR scan to drawing should take under 10 seconds, with minimal typing.
- Feel alive: the AI's live guessing is the emotional core of the product — the UI should make its "thinking" visible and satisfying to watch.
- IEEE-first, carnival-loud: branding should be unmistakable even glanced at from a few feet away (this is a stall attraction, not just an app).
- Replay-friendly: every screen should make "play again" the obvious next action.
- Legible outdoors/indoors: carnival lighting varies — favor high contrast over subtle gradients.

2. SCREEN-BY-SCREEN BREAKDOWN

2.1 Landing Page (post QR scan)

- IEEE logo prominent at top.
- Short, punchy one-line hook (e.g., "Draw it. Beat the AI. Top the board.").
- Single name input field + "Start" button — no other fields, no scroll needed on a phone screen.
- Optional: small live counter ("XXX games played today") for social proof.

2.2 Word Selection

- Three word options shown as large, tappable cards.
- Optional difficulty tag per word (Easy/Medium/Hard) if useful — keep visually minimal (a small dot or label, not a badge-heavy UI).
- Selecting a word immediately transitions to countdown (no separate "confirm" step).

2.3 Countdown

- Full-screen 3-2-1 countdown, bold numerals, IEEE color palette.
- Word to draw shown clearly above/below the countdown so the participant is already thinking about it.

2.4 Drawing Canvas

- Canvas takes up the majority of the screen — this is the core interaction, don't crowd it with chrome.
- Persistent header: word being drawn + round timer (visibly counting down).
- Live AI guess shown as a small, clearly-labeled strip (e.g., "AI thinks: ____") that updates without jarring layout shifts.
- Minimal drawing tools: pen only is sufficient for v1; a "clear canvas" icon is a nice-to-have, not

required.

- Timer nearing zero should have a subtle urgency cue (color shift), not a jarring one (no flashing/strobing).

2.5 Results Screen

- Immediate, unambiguous outcome at the top: "AI got it!" or "Time's up!" — big, celebratory or encouraging tone either way (never make losing feel punishing; keep tone playful).
- Time taken (if correct) and current leaderboard rank shown clearly.
- Two clear actions: "Play Again" (primary) and "View Leaderboard" (secondary).

2.6 Leaderboard (Stall Display)

- Designed for a larger screen viewed from a short distance, not just mobile.
- Top section: headline stats (fastest overall guess, most games played) as large, glanceable numbers.
- Ranked list below: name, score/rank, key stat (successful guesses / best time).
- Auto-refreshing — no manual reload needed; subtle animation on rank changes helps catch the eye of passersby.
- IEEE branding treatment consistent with the rest of the app, but scaled for screen distance (larger type, simplified layout).

3. IEEE BRANDING APPLICATION

- Color palette: primary IEEE blue as the dominant brand color, used for headers, primary buttons, and the leaderboard frame; neutral background (white/light gray) to keep the drawing canvas legible and the AI guess text readable.
- Logo: placed consistently top-left or top-center across all screens (landing, game, results, leaderboard) — never omitted, even on the drawing screen (keep it small/unobtrusive there).
- Typography: a clean, modern sans-serif for all UI text — avoid decorative/script fonts; this is a technical showcase, so the type should read as precise and modern (in the spirit of contemporary product design — Linear/Stripe/Apple-style restraint rather than a "fun carnival poster" font treatment).
- Iconography: minimal, line-based icons rather than flat cartoon-style icons, to keep the aesthetic aligned with a modern product rather than a template-feel game.

4. MICROCOPY GUIDELINES

- Keep all copy short and confident — this is glanced at, not read.
- Encouraging tone on loss ("So close! Try another word" rather than "You lost").
- Celebratory but not over-the-top on wins ("Nailed it in 12s!" rather than excessive exclamation marks/emoji).
- Leaderboard labels should be self-explanatory without a legend (e.g., "Fastest Guess" not "FG

(avg, s)").

5. RESPONSIVE / DEVICE NOTES

- Primary target: mobile portrait orientation (this is how it'll be used at the stall).
- Canvas and touch drawing must work reliably across iOS Safari and Android Chrome — test both explicitly before the event.
- Leaderboard route should separately support a landscape, larger-screen layout for the stall display, sharing the same design language but with a distinct layout (not just a stretched mobile view).

6. ACCESSIBILITY NOTES

- Maintain sufficient color contrast between IEEE blue, text, and backgrounds (test against WCAG AA where feasible, given the short build timeline).
- Ensure tap targets (word cards, buttons) are large enough for quick, casual use in a crowded, possibly distracting environment.
- Avoid relying on color alone to indicate correct/incorrect state — pair with clear text/iconography.

Event Ops & Launch Playbook

DOODLEBOT

Event Ops & Launch Playbook

IEEE Ahmedabad University Student Branch — Club Carnival

1. PURPOSE OF THIS DOC

The PRD, Architecture, and UX/UI docs cover what DoodleBot is and how it's built. This doc covers what happens on the ground — logistics, staffing, and the pre/during/post-event checklist — so the build team and the stall volunteers are aligned on execution.

2. PHYSICAL SETUP

- QR code posters placed at high-traffic points around the venue (entrances, adjacent stalls, main walkway) — not just at the IEEE stall itself, to pull people in from a distance.
- One dedicated screen/monitor at the stall running the Leaderboard route continuously (landscape layout).
- Stable internet connection at the stall — confirm venue Wi-Fi capability in advance; carry a mobile hotspot as backup.
- Signage at the stall itself reinforcing "Scan. Draw. Beat the AI." alongside the QR code, for people already at the stall.

3. STAFFING

- 1–2 volunteers at the stall at all times during carnival hours to:
 - Guide first-time participants through the QR scan
 - Encourage replay and explain the leaderboard
 - Handle basic troubleshooting (e.g., "my drawing isn't loading")
- 1 technical owner (on-call, not necessarily stationed at the stall) responsible for:
 - Monitoring backend/leaderboard uptime
 - Restarting services if something breaks
 - Handling any last-minute fixes

Recommend a simple shift schedule if Club Carnival spans multiple days or long hours, so volunteer attention stays fresh.

4. PRE-EVENT CHECKLIST

- [] Word bank finalized and tested against the AI model for recognition reliability
- [] Full flow tested end-to-end on both iOS Safari and Android Chrome

- [] Load test completed at expected concurrency (20–30 simultaneous sessions)
- [] Leaderboard ranking formula finalized and verified with sample data
- [] IEEE branding assets (logo, colors, palette) applied and reviewed across all screens
- [] QR code posters printed and distribution locations planned
- [] Backup internet (hotspot) arranged
- [] Stall display screen and mount/stand arranged
- [] Volunteer shifts assigned and briefed
- [] Deployment frozen 24–48 hours before the event except for critical fixes

5. DURING-EVENT MONITORING

- Periodically check the leaderboard is updating live (visible confirmation that the backend is healthy).
- Watch for repeated failed sessions (e.g., participants reporting the AI never guesses) — may indicate a word bank or model issue requiring a quick word-list adjustment.
- Keep a lightweight incident log (what broke, when, how it was resolved) for post-event review.

6. POST-EVENT

- Export/save final leaderboard data as a record of engagement (useful for the committee report and future proposals).
- Debrief with volunteers: what confused participants, what drove the most repeat play, what to change next time.
- Archive word bank performance notes (which words the AI struggled with) to improve future iterations.

7. SUCCESS REVIEW CRITERIA (TIE BACK TO PRD METRICS)

At the end of the event, assess against the PRD's success metrics:

- Total unique participants vs. the 500–600 footfall target
- Games-per-participant ratio (repeat play signal)
- Peak concurrency handled without degradation
- Qualitative volunteer feedback on stall engagement and crowd-pulling effect

This review should feed directly into the committee report and any future-iteration proposal.